

expected data in the model, or in case the model receives several samples that is not a multiple of the specified batch size. The arguments taken are `x`, `batch_size`, `verbose`, `steps` and `callbacks`.

5. **train_on_batch:** This method runs one gradient update on a single batch of training data. It returns a scalar training loss if the model has no metrics and a single output or a list of scalars in case the model has multiple outputs and metrics. It takes `x`, `y`, `sample_weight` and `class_weight` as its arguments.
6. **predict_on_batch:** This function returns the predictions for a single batch of samples in the form of numpy array(s). It takes the input sample `x` as its argument.
7. **test_on_batch:** This function tests the model on a single batch of sample data. It returns scalar test loss if the model has no metrics and a single output or a list of scalars in case the model has multiple outputs and metrics. It takes `x`, `y`, and `sample_weight` as its arguments.
8. **Get_layer:** It retrieves a layer based on its unique name or index. In case both name and index are provided then index takes precedence. It will give an error if an invalid layer name or index are specified. The indices traverse the graph horizontally.

Working with a sequential model

To begin working with a sequential model first, we need to define the architecture of the model by defining a series of layers on top of one another. Various layers can be a convolutional layer, maxpool layer, activation layer, dense layer or the dropout layer. A summary of the model can be seen using the `model.summary()` function that also displays the number of training parameters.

The model needs to know the expected dimensions of the input parameter to compile further. Thus, the first layer in a sequential model should be given the shape using the `input_shape` argument. Then we need to compile our model using the `compile` method as discussed above. Further, the model needs to be trained on the dataset. It is usually trained using the `fit()` method. The final step is to make any predictions that are done using the `predict()` method.

To summarise, along with the sequential model and its pros and cons, this chapter discusses the various methods of the sequential models such as `compile`, `fit`, `evaluate` and `predict` along with their parameter description and a list of commonly occurring errors causes. 